Banco de programación y debug remoto para STM32

BeagleBoard-xM · OpenOCD (SWD) · USB DFU · acceso por Tailscale

Documento de trabajo — actualizado al 29/09/2026

Estado: aplicación terminada y probada en modo simulación; puesta en marcha de la placa en curso.

Resumen. Una BeagleBoard-xM sin pantalla ni teclado, ubicada en un banco de laboratorio, funciona como programador y depurador remoto de microcontroladores STM32. Desde el navegador de una PC en otro lugar, conectada por una VPN Tailscale, se sube el firmware, se programa el chip por ST-Link (SWD) o por el bootloader USB DFU de ST, se depura con GDB, se ve la salida en vivo y un monitor serie, y se controla la placa (apagar / reiniciar).

Contenido

1. La idea
2. Arquitectura del sistema (diagrama de bloques)
3. Arquitectura de software (diagrama de bloques)
4. Métodos de programación: SWD y USB DFU
5. Seguridad
6. Configuración
7. Desarrollo y pruebas sin hardware
8. Puesta en marcha de la BeagleBoard-xM (bitácora)
9. Próximos pasos

Anexo A. Estructura del repositorio

Anexo B. Comandos útiles

1. La idea

El problema. Los microcontroladores con los que se trabaja están conectados en un banco de pruebas de laboratorio, pero quien los programa y depura está en otro lugar. Sin una herramienta remota, cada cambio de firmware exige estar físicamente frente al banco.

La propuesta. Convertir una BeagleBoard-xM en un *banco de programación y debug remoto*: la placa queda conectada al programador (ST-Link) y al STM32, sólo con alimentación de 5 V y cable de red, y ofrece una aplicación web accesible únicamente a través de una VPN (Tailscale). Todo lo que se haría con el programador enchufado a la PC se hace desde el navegador.

Objetivos

- **Programar** cualquier STM32: flashear (program + verify + reset), sólo verificar, borrar la flash completa, reset, halt y run.
- **Dos métodos de conexión** seleccionables por chip o por operación: ST-Link por SWD con OpenOCD (programación y debug) y USB DFU con dfu-util (sólo programación, sin debug).
- **Depurar remotamente** con GDB: arm-none-eabi-gdb, VS Code (Cortex-Debug) o STM32CubeIDE.
- **Gestionar el firmware**: subir .elf/.hex/.bin arrastrando al navegador o sincronizar una carpeta con Syncthing.
- **Ver en vivo** la salida de OpenOCD/dfu-util, un historial de operaciones y un monitor serie.
- **Controlar la placa**: estado del sistema, apagar y reiniciar con confirmación.
- **Soporte configurable**: agregar un chip nuevo es agregar una entrada a un archivo YAML.
- **Seguridad**: sin exposición a internet, login, y ningún comando armado con texto del usuario.

Requisitos y restricciones

Aspecto	Detalle
Placa	BeagleBoard-xM: TI DM3730 (Cortex-A8, ARMv7, armhf), 512 MB de RAM, arranque desde microSD, 4 puertos USB host, Ethernet. Sin pantalla, teclado ni mouse (headless).
Programador	ST-Link V2 / V2-1 / V3, integrado en Nucleo/Discovery o externo, por USB.
Chips actuales	STM32H503CBT6 (Cortex-M33, serie H5) y STM32H723VGT6 (Cortex-M7, serie H7).
Red	La PC y la placa se conectan por Tailscale. No se abren puertos en el router.
Liviandad	Pocos recursos: backend Python (FastAPI), frontend sin compilación ni Node.
Operación	Servicios con systemd; arranque automático; usuario sin privilegios.

2. Arquitectura del sistema

La figura 1 muestra los bloques del sistema completo. A la izquierda, la PC del usuario; en el medio, el túnel cifrado de Tailscale; a la derecha, la BeagleBoard-xM con los procesos que corren en ella y el hardware de programación conectado a sus puertos USB.

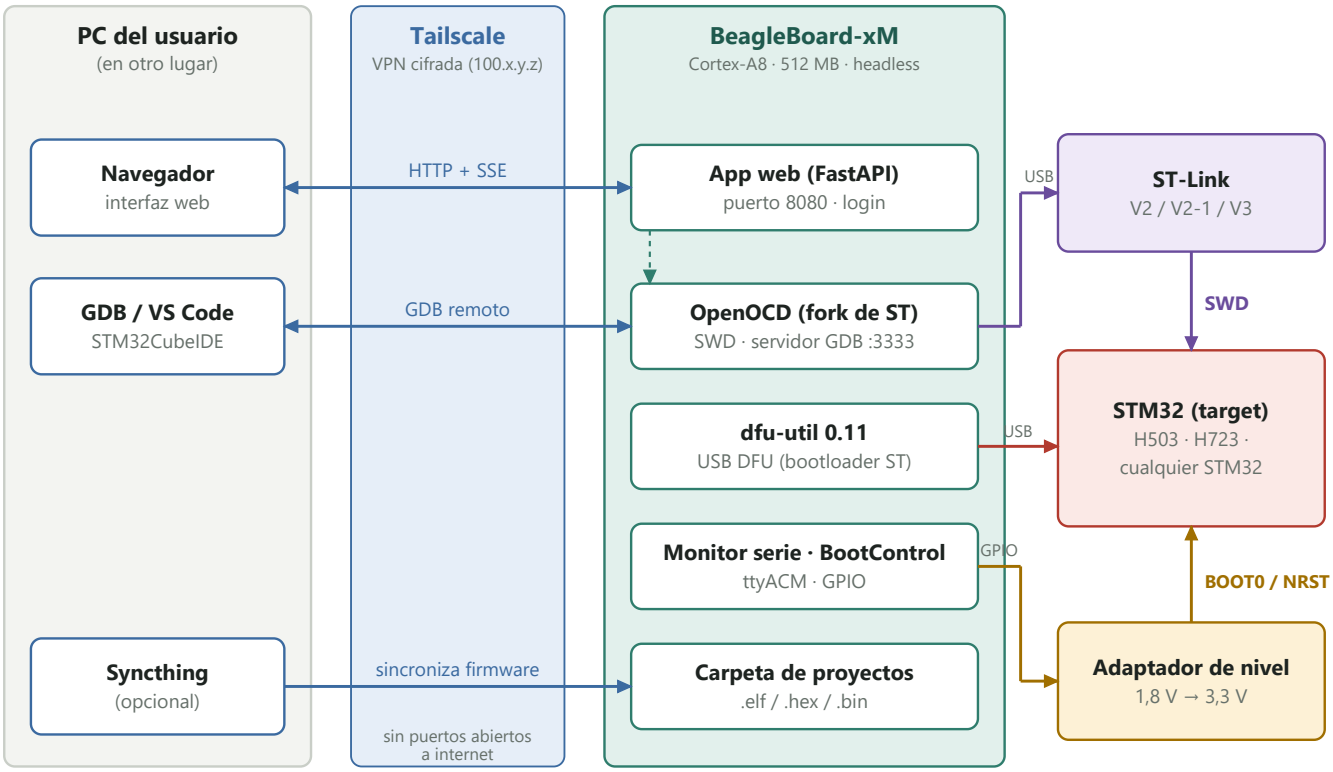


Figura 1 — Diagrama de bloques del sistema. Las flechas azules pasan por la VPN; las de la derecha son conexiones físicas (USB, SWD y GPIO).

Bloques

- **PC del usuario**: navegador para la interfaz web y cliente GDB (arm-none-eabi-gdb, VS Code o CubeIDE). Opcionalmente, Syncthing para sincronizar la carpeta de compilación.
- **Tailscale**: VPN de malla (WireGuard). La placa recibe una IP 100.x.y.z accesible sólo desde los equipos de la cuenta; no hay que abrir puertos en el router.
- **App web**: servicio systemd en el puerto 8080. Orquesta todo lo demás.
- **OpenOCD** (fork oficial de ST, el de STM32CubeIDE): programa por SWD y, en modo servidor, expone el puerto GDB 3333. Se compila en la placa porque el de los repositorios es viejo para H5/H7.
- **dfu-util**: programa por USB a través del bootloader de sistema del STM32 (VID:PID 0483:df11).
- **ST-Link**: conectado por USB a la placa y por SWD (SWDIO, SWCLK, GND, NRST) al STM32.
- **USB del propio STM32** (PA11/PA12): usado para DFU. En una Nucleo es un conector distinto al del ST-Link, así que pueden estar los dos enchufados.
- **Adaptador de nivel** (opcional, para BootControl por GPIO): los GPIO de la BeagleBoard-xM son de 1,8 V y BOOT0/NRST del STM32 son de 3,3 V; no se pueden conectar directo.

3. Arquitectura de software

Dentro de la placa, la aplicación está organizada en capas (figura 2). Cada pedido del navegador atraviesa el middleware de seguridad, llega a la API y se valida en el servicio central (*Bench*) antes de construir cualquier comando. Los procesos externos se lanzan siempre con una lista de argumentos, nunca a través de una shell.

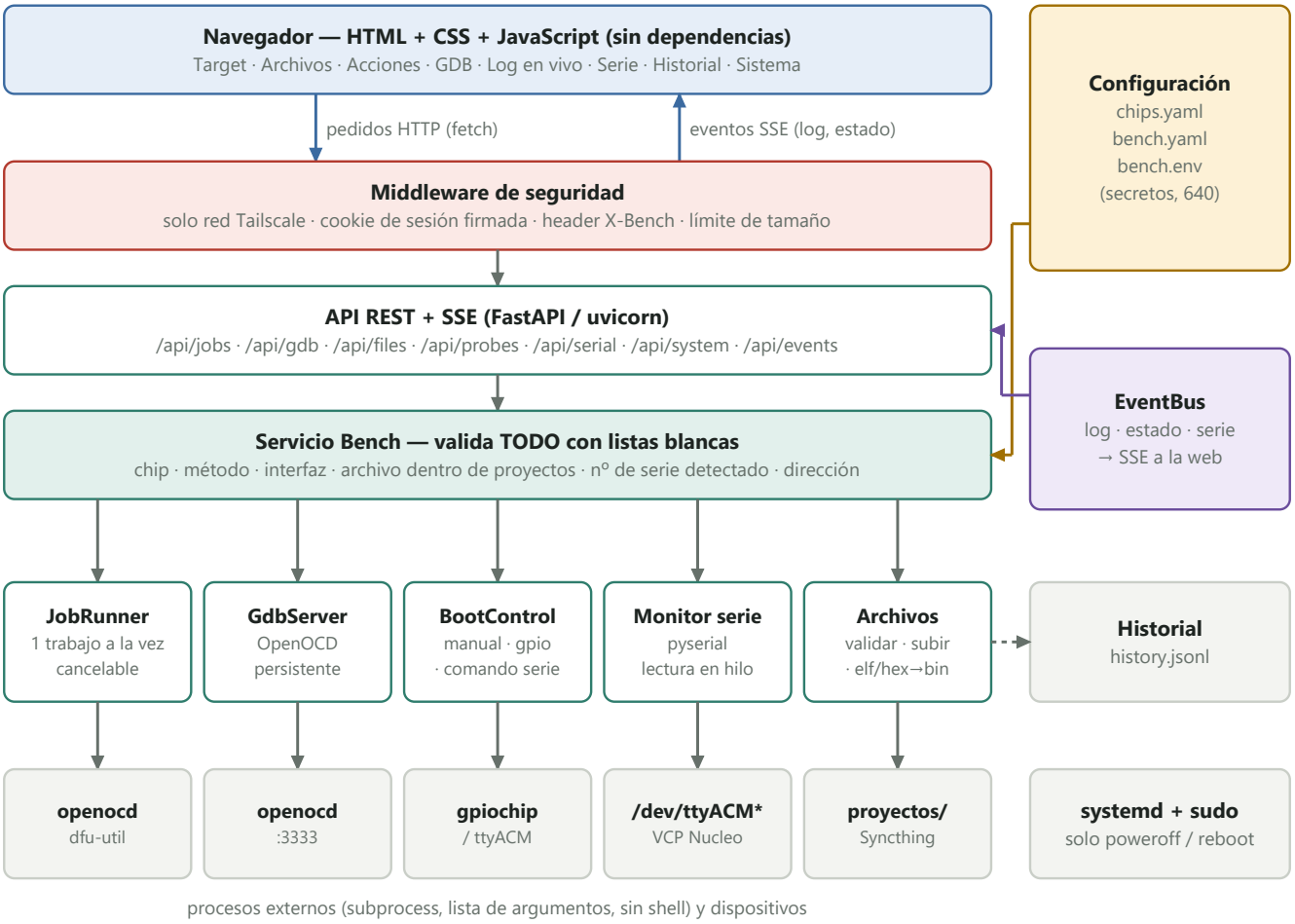


Figura 2 — Bloques de software en la BeagleBoard-xM.

Decisiones técnicas

Decisión	Motivo
FastAPI + uvicorn (sin extras)	Asincronismo nativo para SSE, procesos largos y el monitor serie. ~40–60 MB de RAM. Sin extensiones que compilar en armhf.
Server-Sent Events (SSE)	El log es de una sola vía; SSE no necesita librerías y el navegador reconecta solo. El envío por serie usa POST.
Frontend HTML + CSS + JS sin dependencias	Sin Node ni compilación; se sirve desde el mismo backend. Dimensionado para 1366×768.
Contraseña con scrypt (hashlib)	KDF recomendado por OWASP, en la biblioteca estándar: no hay que compilar bcrypt/argon2 en armhf.
Un trabajo a la vez (409 si está ocupado)	Nunca se pisan dos sesiones del programador; encolar flasheos a ciegas es más riesgoso.
Procesos con Popen + hilo lector	Mismo código en Linux (placa) y Windows (desarrollo), salida línea a línea al log en vivo.
Modo simulación (BENCH_MOCK=1)	Simuladores de OpenOCD, dfu-util, ST-Link, DFU, serie y sistema para desarrollar y probar sin hardware.

4. Métodos de programación: SWD y USB DFU

Cada chip declara en **chips.yaml** qué métodos soporta y sus parámetros. La interfaz sólo habilita los métodos disponibles y, con DFU, deshabilita lo que el bootloader no puede hacer, explicando el motivo.

Acción	ST-Link (SWD) — OpenOCD	USB DFU — dfu-util
Flashear	program … verify reset exit	-s 0x08000000:leave -D firmware.bin
Verificar	halt · verify_image · resume	leer con -U y comparar
Borrar flash	flash erase_sector <banco> 0 last	-s 0x08000000:mass-erase:force
Reset / Halt / Run	sí	no (sin acceso al núcleo)
Servidor GDB	sí (puerto 3333)	no
.elf / .hex	directo	se convierten a .bin en la placa

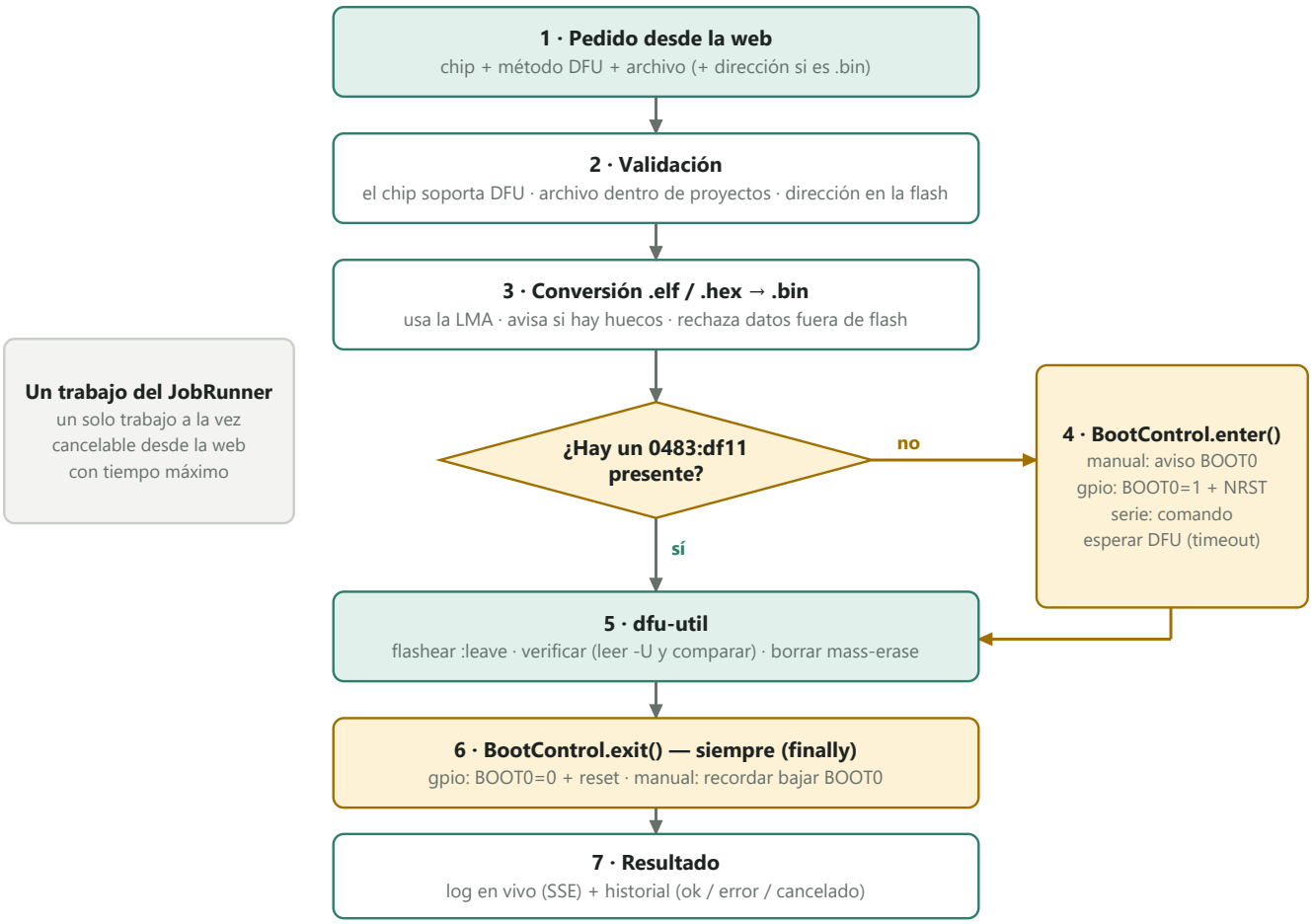


Figura 3 — Flujo de una operación por USB DFU.

Ejemplo de comando construido por la app para flashear un STM32H723 (lista de argumentos; los valores entre llaves son literales en Tcl):

```
openocd -s /usr/local/share/openocd/scripts
        -f interface/stlink-dap.cfg  -c "transport select dapdirect_swd"
        -c "adapter serial 066DFF485550755187121832"
        -f target/stm32h7x.cfg       -c "adapter speed 4000"
        -c "program {/srv/stm32-bench/proyectos/demo/Debug/demo.elf} verify reset exit"
```

> **Hallazgo al revisar el fork de ST (tag openocd-cubeide-v2.2.0):** la CPU del STM32H5 está en el AP 1 del DAP, por lo que el modo HLA (`interface/stlink.cfg`) no sirve; hace falta `stlink-dap.cfg` con `transport select dapdirect_swd`, que ese archivo no elige solo. El ST-Link se selecciona con `adapter serial`.

Backends de BootControl (cómo se entra al bootloader)

Backend	Cómo funciona	Requisitos
manual	La web muestra "Poné BOOT0 en alto y reseteá la placa" con cuenta regresiva y sigue sola cuando aparece el dispositivo DFU.	Una persona en el banco.
gpio	La placa maneja BOOT0 y NRST con libgpiod (/dev/gpiochipN, nunca sysfs): BOOT0=1, pulso de NRST, espera DFU; al final BOOT0=0 y reset.	Adaptador 1,8 V → 3,3 V (NPN en NRST, traductor no inversor en BOOT0).
serial_command	Envía un comando configurable (p. ej. "BOOTLOADER\n") por el puerto serie al firmware, que salta al bootloader de sistema.	Soporte en el firmware (dirección de System Memory según AN2606).

5. Seguridad

La aplicación puede ejecutar comandos en la placa, así que la seguridad se diseñó en capas:

- **Red**: acceso sólo por Tailscale; no se abren puertos en el router. Además la app rechaza conexiones que no vengan de las redes de Tailscale (100.64.0.0/10, fd7a:115c:a1e0::/48) o de localhost.
- **Autenticación**: usuario y contraseña; hash scrypt; cookie de sesión firmada, HttpOnly, SameSite=Strict, con vencimiento; límite de intentos fallidos por IP. Sin credenciales en el código: van en /etc/stm32-bench/bench.env (permisos 640).
- **CSRF**: toda petición que modifica algo exige el header X-Bench (un sitio ajeno no puede agregarlo sin CORS, que no está habilitado).
- **Inyección de comandos**: subprocess con lista de argumentos y sin shell; chips, interfaces y backends sólo desde los YAML; rutas resueltas dentro del directorio de proyectos (sin ".." ni symlinks hacia afuera); lista blanca de caracteres en los nombres, porque los argumentos -c de OpenOCD son código Tcl y una ruta con [ ] { } $ podría inyectar Tcl.
- **Privilegios**: el servicio corre como usuario stm32bench (grupos plugdev, dialout, gpio). sudo sólo permite poweroff y reboot sin argumentos, validado con visudo -c.
- **GDB**: el puerto 3333 no tiene autenticación, por eso escucha sólo en la IP de Tailscale y OpenOCD se lanza con telnet y Tcl deshabilitados. Se recomiendan ACL de Tailscale.
- **Cabeceras HTTP**: Content-Security-Policy estricta, X-Frame-Options DENY, nosniff, no-referrer.

6. Configuración

Los archivos viven en /etc/stm32-bench/ (en el repositorio, en config/). Agregar un chip es agregar una entrada:

```
- id: stm32h723vg
  name: STM32H723VGT6
  family: H7 (Cortex-M7)
  flash: { base: 0x08000000, size_kb: 1024, sector_kb: 128 }
  default_method: swd
  methods:
    swd:
      target_cfg: target/stm32h7x.cfg      # relativo a los scripts de OpenOCD
      interface: stlink-dap                # definida en bench.yaml
      adapter_speed_khz: 4000
      flash_banks: [0]                     # bancos a borrar
    dfu:
      vid_pid: "0483:df11"
      alt: 0                               # "Internal Flash"
```

Archivo	Contenido
chips.yaml	Catálogo de chips: H503, H723 y ejemplos F1, F4, L4, G0, G4. Al arrancar se verifica que los .cfg existan en el OpenOCD instalado.
bench.yaml	Binarios y timeouts de OpenOCD/dfu-util, interfaces (stlink = HLA, stlink-dap = dapdirect), puerto y bind del GDB, tamaño máximo de subida, baudrates, redes permitidas, BootControl.
bench.env	Usuario, hash de la contraseña, clave de sesión y rutas. Lo genera install.sh.

7. Desarrollo y pruebas sin hardware

Para desarrollar en la PC (Windows) sin la placa, el **modo simulación** (BENCH_MOCK=1) reemplaza OpenOCD y dfu-util por simuladores que se ejecutan como procesos reales, así el camino de ejecución y el log en vivo son los mismos que en la placa. También simula dos ST-Link, la aparición del dispositivo DFU al entrar al bootloader, un puerto serie y los datos del sistema. Se crean proyectos de ejemplo, incluidos casos de error (hueco entre segmentos, datos en flash externa, verificación fallida).

```
powershell -ExecutionPolicy Bypass -File scripts\dev_run.ps1   # http://127.0.0.1:8080
```

Pruebas automáticas

156 tests con pytest, todos pasando. Cubren:

- Validación de archivos: nombres, extensiones, tamaño, contenido (ELF ARM de 32 bits, Intel HEX), path traversal y symlinks.
- Construcción de comandos de OpenOCD y dfu-util: orden de argumentos, direcciones, bloqueo de inyección Tcl, parseo de dfu-util -l.
- Carga y validación de chips.yaml y bench.yaml; detección de .cfg faltantes.
- Selección de método según el chip y capacidades de cada método.
- Conversión .elf/.hex → .bin: uso de la LMA, huecos, datos fuera de flash, solapamientos.
- BootControl: secuencia de GPIO con un driver falso, manual, comando serie, timeout, cancelación y flujo DFU completo con el simulador.
- API: login, redes permitidas, CSRF, trabajos concurrentes (409), bloqueo por servidor GDB activo, monitor serie, apagado.

Pruebas en el navegador

Con la app en modo simulación se recorrieron los flujos principales: flasheo por SWD con log en vivo, flasheo por DFU con el diálogo “Poné BOOT0 en alto…” (que se cierra solo al aparecer el dispositivo) y el aviso de segmentos discontinuos, servidor GDB (muestra IP:3333) y el pedido de detenerlo al flashear, monitor serie, historial y estado del sistema, a 1366×678.

Problemas encontrados y corregidos en esas pruebas:

- El diálogo de confirmación dependía del evento *close* del elemento <dialog>, que el navegador no emite si la pestaña no se está dibujando: ahora se resuelve desde los botones.
- El navegador usaba CSS/JS viejos de la caché: los archivos estáticos ahora se sirven con Cache-Control: no-cache (revalidación con ETag).
- La lista de archivos repetía el nombre del proyecto como subcarpeta.

8. Puesta en marcha de la BeagleBoard-xM (bitácora)

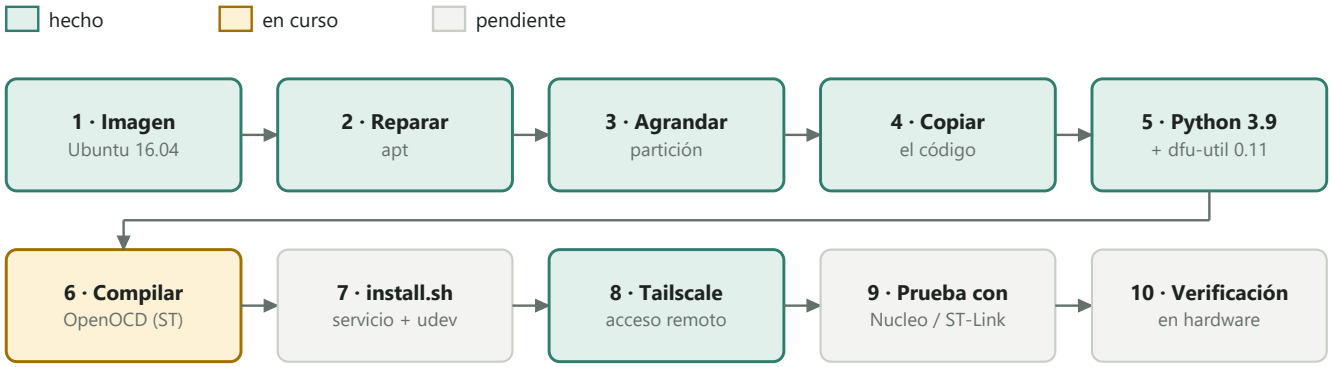


Figura 4 — Pasos de la puesta en marcha y su estado al 29/09/2026.

8.1 Estado inicial encontrado

Elemento	Encontrado
Sistema	Ubuntu 16.04 LTS, imagen de consola de rcn-ee (Robert C. Nelson) del 23/01/2017.
Kernel	4.9.5-armv7-x4 (armv7l).
Memoria	483 MB utilizables, sin swap.
Almacenamiento	microSD de 3,7 GB, pero la partición raíz ocupaba sólo 1,6 GB (474 MB libres).
Python	3.5.2 (la app necesita 3.9 o superior).
Red	eth0 192.168.0.27/24 (LAN) y usb0 192.168.7.2 (red por USB de la imagen).
apt	Instalación de paquetes interrumpida y un repositorio https sin soporte instalado.

8.2 Decisión: mantener Ubuntu 16.04

Se evaluó instalar un sistema más nuevo, pero la página oficial de imágenes de BeagleBoard.org ya no publica imágenes para la xM; una Debian nueva habría que armarla a partir del sistema base de rcn-ee más kernel y bootloader. Actualizar en el lugar (16.04 → 18.04 → 20.04 → 22.04) es lento y riesgoso con un kernel no estándar. Como los repositorios de 16.04 siguen respondiendo, se decidió conservar el sistema y agregar un Python 3.9 compilado aparte (la última versión que compila con el OpenSSL 1.0.2 de 16.04). El acceso sólo por Tailscale compensa la falta de actualizaciones de seguridad.

8.3 Acceso y reparación de apt

Con monitor y teclado se ingresó con el usuario por defecto de la imagen (indicado en la pantalla de login) y luego se trabajó por SSH desde la PC, para copiar y pegar comandos. Se recomienda cambiar la contraseña de fábrica con `passwd`.

```
ssh ubuntu@192.168.0.27
sudo dpkg --configure -a
sudo apt install -y apt-transport-https ca-certificates
sudo apt update          # OK: ports.ubuntu.com (xenial) y repos.rcn-ee.com responden
```

8.4 Agrandar la partición a toda la microSD

```
sudo apt install -y cloud-guest-utils tmux
sudo growpart /dev/mmcblk0 2        # CHANGED: partition=2 ... new: size=7690207
sudo resize2fs /dev/mmcblk0p2       # redimensionado en línea, sin reiniciar
df -h /                             # 3,6 GB de tamaño, 2,4 GB libres
```

8.5 Copia del código a la placa

El repositorio todavía no está publicado; se empaquetó en la PC y se copió por SSH (219 KB).

```
# En la PC (PowerShell)
tar --exclude=.venv --exclude=dev-data --exclude=.git -czf $env:USERPROFILE\Desktop\stm32-bench.tgz
    -C $env:USERPROFILE\Desktop\Debugger .
scp $env:USERPROFILE\Desktop\stm32-bench.tgz ubuntu@192.168.0.27:~
# En la placa
mkdir -p ~/stm32-bench && tar -xzf ~/stm32-bench.tgz -C ~/stm32-bench && cd ~/stm32-bench
```

8.6 Preparación del sistema (completada)

Se agregó el script `scripts/prepare_ubuntu1604.sh`, que se ejecuta dentro de tmux (si se corta el SSH, el trabajo sigue; se retoma con `tmux attach -t prep`):

- Instala las dependencias de compilación y crea un swap temporal de 512 MB.
- Descarga Python 3.9.25, verifica su SHA-256, lo compila e instala en /opt/python3.9 sin tocar el Python del sistema (1 a 2 horas en la xM).
- Compila dfu-util 0.11 en /usr/local (el de 16.04 es 0.8 y no tiene mass-erase), verificando su MD5.
- Borra los archivos de compilación para ahorrar espacio.

```
tmux new -s prep
sudo bash scripts/prepare_ubuntu1604.sh
```

Resultado: terminó en unos 72 minutos (20:02 → 21:14, hora de la placa). Quedaron Python 3.9.25 con OpenSSL 1.0.2g (todos los módulos necesarios cargan) y dfu-util 0.11. Durante la compilación sólo aparecieron advertencias del compilador (variables sin usar, strict-aliasing), sin errores; las advertencias de pip por ejecutarse como root son esperables porque la app usa su propio entorno virtual.

8.7 Tailscale (hecho en paralelo a la compilación)

Desde otra sesión SSH, mientras compilaba Python. La imagen no traía `curl` (se agregó al script de preparación). El repositorio de Tailscale todavía publica paquetes para Ubuntu 16.04 armhf (versión 1.102.4).

```
sudo apt install -y curl
curl -fsSL https://tailscale.com/install.sh | sh
sudo tailscale up --hostname=banco-stm32     # se autoriza abriendo el enlace en la PC
```

En la consola de Tailscale (Machines → banco-stm32 → □ → Disable key expiry) se desactivó el vencimiento de la clave, para que la placa no quede fuera de la red. La placa obtuvo la IP **100.99.235.81** y se verificó el acceso remoto desde la PC con `ssh ubuntu@100.99.235.81`. Desde ahí se usa esa IP (o el nombre banco-stm32) en lugar de la IP de la red local.

Otros ajustes hechos para 16.04: `install.sh` detecta /opt/python3.9, instala pydantic-core sólo como paquete precompilado (se verificó que existe para armv7 y Python 3.9 hasta la versión 2.46.5) y ya no usa piwheels; `build_openocd.sh` clona sólo la versión necesaria y borra el código al terminar. En 16.04 el backend GPIO de BootControl queda pendiente (no hay python3-libgpiod y el kernel 4.9 no tiene la API v2 de GPIO).

9. Próximos pasos

Paso	Comando / acción	Duración estimada
Compilar OpenOCD (fork de ST)	sudo bash scripts/build_openocd.sh	45–90 min
Instalar la app	sudo bash scripts/install.sh (pide usuario y contraseña de la web)	10–20 min
Primera prueba	Nucleo por USB del ST-Link; lsusb | grep 0483; flashear desde la web	—
GDB remoto	arm-none-eabi-gdb → target extended-remote <IP>:3333	—

Verificaciones pendientes en hardware

- Compilación de OpenOCD en 16.04 (gcc 5.4) y presencia de stm32h5x.cfg, stm32h7x.cfg y stlink-dap.cfg.
- Instalación de dependencias Python sin compilar en armv7; arranque del servicio systemd 229.
- Operaciones reales en STM32H503 (stlink-dap) y STM32H723: borrado genérico, verify_image, halt/run.
- Selección por adapter serial con dos ST-Link; números de serie de ST-Link/V2 viejos.
- Servidor GDB en la IP de tailscale0; conexión desde GDB y VS Code.
- DFU real: salida de dfu-util -l, :leave, verificación con -U y mass-erase sin -D.
- Monitor serie con el VCP de la Nucleo; datos del sistema (temperatura del DM3730); apagar/reiniciar.
- Consumo de RAM/CPU de la app en la xM.

Anexo A. Estructura del repositorio

```
app/                backend (FastAPI)
  main.py           aplicación, middleware de seguridad, sesiones, estáticos
  api.py            rutas /api/...
  bench.py          servicio central: valida pedidos y arma trabajos SWD/DFU
  settings.py       variables de entorno y bench.yaml
  chips.py          chips.yaml y verificación de .cfg
  files.py          validación, subida, escaneo y borrado de firmware
  convert.py        .elf/.hex -> .bin para DFU
  probe.py          ST-Link por sysfs y dispositivos DFU
  programmers/      capacidades, comandos OpenOCD y dfu-util, JobRunner, GdbServer
  bootcontrol/      manual, gpio (libgpiod), comando serie, simulado
  serial_monitor.py monitor serie (pyserial)
  system.py         estado del sistema, apagar/reiniciar
  events.py         bus de eventos para SSE
  history.py        historial (JSON Lines)
web/                interfaz (HTML, CSS, JS sin dependencias)
config/             chips.yaml, bench.yaml, bench.example.env
scripts/            build_openocd.sh, prepare_ubuntu1604.sh, install.sh, hash_password.py,
                    dev_run.ps1/.sh, simuladores de OpenOCD y dfu-util
deploy/             unidad systemd, reglas udev, plantilla de sudoers
docs/               este documento y su generador
tests/              156 tests (pytest)
```

Anexo B. Comandos útiles

```
ssh ubuntu@100.99.235.81             # entrar a la placa por Tailscale (o ubuntu@banco-stm32)
tmux attach -t prep                  # volver a una sesión de tmux
systemctl status stm32-bench         # estado del servicio
journalctl -u stm32-bench -f         # log del servicio
sudo systemctl restart stm32-bench   # reiniciar tras editar /etc/stm32-bench/*.yaml
lsusb | grep 0483                    # ST-Link / DFU conectados
df -h / ; free -m                    # espacio y memoria
```